3-1 内存的分配

- 1. 连续分配基础
  - 基本含义
    - **连续分配方式**：给用户程序分配一个连续的内存空间。
    - **逻辑相邻**：程序中代码或数据的逻辑地址相邻。
    - **物理相邻**：体现在内存空间分配时，物理地址也相邻。
    - **装入条件**：要把用户程序装入内存，必须给它分配一定内存空间。
  - 内存保护
    - **保护目标**：程序之间不能互相影响。
    - **上下限寄存器**：CPU 中设置一对上下限寄存器，记录用户进程在主存中的上下限地址。
    - **上下限检查**：CPU 每次访存时，将访问地址与上下限寄存器比较，检查是否越界。
    - **重定位寄存器**：存放最小合法物理内存地址，也就是基地址。
    - **界地址寄存器**：存放合法范围的大小，也称界限地址。
    - **越界处理**：逻辑地址若超过界地址寄存器范围，则产生地址错误。
    - **地址形成**：合法逻辑地址与重定位寄存器相加，得到物理地址。
  - 内存共享
    - **可重入代码**：允许多个进程同时访问，但不允许被任何进程修改的代码，也称纯代码。
    - **共享条件**：只有只读区域适合共享。
    - **不适合共享**：数据区通常不适合被多个进程任意共享修改。
- 2. 单一连续分配
  - 基本方式
    - **适用环境**：单道程序环境。
    - **内存划分**：把内存分为系统区和用户区。
    - **系统区**：供 OS 使用，一般放在内存低地址部分。
    - **用户区**：只装入一道用户程序。
    - **独占特点**：整个用户空间由该用户程序独占。
- 3. 固定分区分配
  - 基本方式
    - **适用环境**：多道程序系统。
    - **划分方式**：把整个用户空间划分成几个固定大小区域。
    - **装入规则**：每个分区只装一个作业。
    - **分区大小**：各区域可以等大，也可以不等大。
  - 分区使用表
    - **作用**：便于分配和回收。
    - **排列方式**：可按分区大小排队。
    - **表项内容**：分区号、分区大小、起始地址、状态。
  - 内部碎片
    - **形成原因**：分区空间已分给程序，但其中一部分未被程序使用。
    - **内部碎片**：已分配分区内部未被使用的空间。
- 4. 动态分区分配
  - 空闲分区表
    - **设置目的**：记录系统中每个空闲分区的情况。
    - **表目单位**：每个空闲分区占用一个表目。
    - **表项内容**：分区号、分区大小、分区起始地址、状态。
  - 首次适应算法
    - **规则**：找到第一个能满足作业大小的空闲分区并分配。
    - **查找起点**：从空闲分区表的开始位置查找。
    - **分割处理**：若空闲分区大于作业需求，则切出需要部分，剩余部分继续作为空闲分区。
  - 邻近适应算法
    - **性质**：对首次适应算法的改进。
    - **规则**：从上次查找结束位置开始继续查找。
    - **分配处理**：找到能满足需求的空闲分区后分配，并更新下次查找位置。
  - 最佳适应算法
    - **规则**：找到最小的、能满足作业大小的空闲分区并分配。
    - **空闲表排列**：空闲分区按容量递增排列。
    - **分割处理**：产生新的空闲分区后，需要重新按大小排序。
  - 最坏适应算法
    - **规则**：找到最大的、能满足作业大小的空闲分区并分配。
    - **空闲表排列**：空闲分区按容量递减排列。
    - **分割处理**：分割后的新空闲分区需要重新调整空闲分区表。
- 5. 伙伴系统
  - 基本规则
    - **分区大小**：所有分区大小均为 `2^k` ， `k` 为正整数。
    - **空闲链表**：所有相同大小的空闲分区，单独设置一个空闲分区双向链表。
    - **需求计算**：需要给进程分配大小为 `n` 的空间时，先找满足 `2^(i-1) < n <= 2^i` 的分区大小。
    - **直接分配**：若大小为 `2^i` 的空闲分区链表中有空闲分区，直接分配。
  - 分裂
    - **找不到目标大小**：若找不到大小为 `2^i` 的空闲分区，就在更大的 `2^(i+1)` 空闲分区链表中查找。
    - **分裂方式**：把较大分区分成两个相同大小的分区。
    - **伙伴**：分裂得到的两个相同大小分区称为一对伙伴。
    - **分配方式**：其中一个分区用于分配，另一个加入对应大小的空闲分区链表。
  - 内部碎片
    - **形成原因**：实际申请空间小于分配到的 `2^i` 分区。
    - **表现**：多出来但已分配给进程的空间成为内部碎片。
- 6. 紧凑技术
  - 外部碎片
    - **形成原因**：动态分区分配后，内存中可能出现许多小的空闲空间。
    - **外部碎片**：难以被分配出去的小空闲空间。
  - 紧凑
    - **作用**：整理外部碎片，把分散空闲空间合并成较大的连续空间。
    - **支持条件**：需要动态重定位寄存器支持。
    - **代价**：整理过程费时。
- 7. 分区分配操作
  - 分配内存
    - **查找分区**：根据系统分配算法，在空闲分区表中找到对应分区。
    - **空间足够且更大**：若进程需要空间小于空闲分区空间，则分割空闲分区。
    - **剩余空间**：多余部分继续留在空闲分区表中。
    - **分配部分**：满足需求的部分分配给申请进程。
  - 回收内存
    - **触发时机**：进程运行完毕释放内存。
    - **插入位置**：系统根据回收区首址，在空闲表中找到对应插入点。
    - **合并判断**：根据回收区是否与前后空闲分区相邻，选择是否合并表项。
    - **相邻合并**：若回收区与前后分区相连，需要合并成更大的空闲分区。
- 8. 分页存储
  - 引入原因
    - **连续分配问题**：会形成许多碎片。
    - **紧凑代价**：虽然能通过紧凑拼出较大空间，但开销大。
    - **离散思想**：把进程切成小块，分散装入许多不相邻区域。
    - **效果**：充分利用内存空间，无须紧凑。
  - 页与物理块
    - **页或页面**：用户程序被分割成的固定大小区域。
    - **物理块或页框**：内存被分割成的固定大小区域。
    - **装入关系**：页面可以装入任意物理块。
    - **映射需求**：每一页可能装入任意块，因此需要页表记录映射关系。
  - 分页地址结构
    - **逻辑地址组成**：页号 + 页内偏移量。
    - **页号作用**：指出要访问的数据位于哪一页。
    - **页内偏移量作用**：指出具体需要页内哪一个字节或单元。
    - **页大小影响**：页大小决定页内偏移量位数。
  - 页表
    - **页表组成**：页表由多个页表项组成。
    - **页表项作用**：记录某个页面存放在哪个物理块中。
    - **页号存储**：页号顺序递增时，可以省略不存。
    - **其他字段**：页表中还可能有有效位、算法位等字段。
  - 分页地址变换
    - **地址拆分**：分页地址变换机构把逻辑地址分成页号和页内地址。
    - **越界检查**：先用页号与页表长度比较。
    - **越界中断**：页号大于页表长度时，说明访问超越进程地址空间，产生越界中断。
    - **查找页表项**：用 `页表始址 + 页号 * 页表项长度` 得到页表项位置。
    - **获得块号**：从页表项中取出物理块号。
    - **形成物理地址**：物理块号与页内地址组合，完成逻辑地址到物理地址的变换。
  - 快表 TLB
    - **问题**：页表在内存中时，一次取数据通常需要访问两次内存。
    - **第一次访存**：访问内存中的页表，取出块号。
    - **第二次访存**：根据物理地址访问内存，取出数据。
    - **快表思想**：效仿 Cache，增加快表 TLB，保存当前访问的常用页表项。
    - **TLB 命中**：CPU 给出地址后，取出页号与 TLB 中页号比较，匹配成功即可直接得到块号。
    - **TLB 命中效果**：少一次访问内存的时间。
    - **TLB 未命中**：继续访问内存中的页表寻找页表项。
    - **写入 TLB**：找到后把物理块号送入地址寄存器，同时把该页表项存入快表。
    - **TLB 已满**：根据算法替换掉一个表项。
- 9. 二级和多级页表
  - 引入原因
    - **大逻辑地址空间**：现代计算机支持非常大的逻辑地址空间。
    - **页表过大**：页表会变得很大，占用大量内存空间。
    - **连续空间压力**：单级页表可能要求占用较大的连续内存空间。
    - **处理方式**：把页表再次分页，减少对连续内存空间的需求。
  - 二级页表结构
    - **页目录表**：顶级页表，记录二级页表所在物理块号。
    - **二级页表**：离散存放，记录数据页面所在物理块号。
    - **数据页面**：存放具体数据，也离散存放。
    - **常驻内存**：页目录表常驻内存。
  - 二级地址结构
    - **逻辑地址组成**：页目录号 + 页号 + 页内偏移。
    - **页目录号作用**：查询顶级页表，得到二级页表所在物理块号。
    - **页号作用**：在二级页表中找到目标页面的物理块号。
    - **页内偏移作用**：锁定目标页面中的具体数据。
- 10. 分段存储
  - 基本思想
    - **分页特点**：通过硬件机制实现，对用户完全透明。
    - **分段特点**：考虑用户和程序员的需要。
    - **逻辑划分**：把用户进程的逻辑地址空间划分为大小不等的段。
    - **常见段**：主程序段、子程序段、栈段、数据段。
    - **段内编址**：每段都从 0 开始编址。
    - **内存分配**：每段在内存中分配一段连续地址空间。
    - **连续性**：段内要求连续，段间不要求连续。
    - **地址空间**：进程地址空间是二维的。
  - 分段地址结构
    - **逻辑地址组成**：段号 + 段内地址。
    - **段号作用**：指出访问哪个段。
    - **段内地址作用**：指出访问该段内的哪个位置。
  - 段表
    - **段表项内容**：段号、段长、基址。
    - **段长作用**：用于段内地址越界检查。
    - **基址作用**：指出该段在主存中的起始位置。
    - **地址变换**：段表始址定位段表项，段内地址与段基址相加得到物理地址。
    - **访存次数**：段表在内存中时，访问一个数据需要访问两次内存。
    - **加速方法**：增加一个存储器保存最近常用段表项。
- 11. 段页式存储
  - 基本思想
    - **组合方式**：先分段，再分页。
    - **段名**：先把用户程序分成段，并为每个段赋予段名。
    - **段内分页**：再把每个段分成页。
    - **综合特点**：结合分段和分页。
  - 地址结构
    - **作业地址结构**：段号 `S` + 段内页号 `P` + 页内地址 `W` 。
    - **段号作用**：定位段。
    - **段内页号作用**：定位该段内的页面。
    - **页内地址作用**：定位页内具体单元。
  - 地址变换
    - **段表寄存器**：保存段表长度和段表始址。
    - **段号检查**：若段号大于段表长度，产生越界中断。
    - **查段表**：用段号找到对应段表项。
    - **段表项内容**：页表长度、页表始址等。
    - **查页表**：用段内页号查找页表，得到物理块号。
    - **形成物理地址**：物理块号与页内地址组合成物理地址。